DISEÑO DE BASES DE DATOS II

GUÍA DE ESTUDIO Y DEFENSA

GESTIÓN DE PASAJES

MongoDB documental

Modelado, ObjectId, índices, consultas,
rendimiento y política de respaldo

9 colecciones | 35 índices | 15 My Queries | MongoDB 8.0.32

Sistema local de demostración para una empresa de colectivos de larga distancia

Edición 2026

Resumen ejecutivo

gestionpasajes es una solución documental para administrar recorridos, viajes, asientos por tramo, reservas temporales, pasajes vendidos y seguimiento operativo de una empresa de colectivos de larga distancia. El diseño parte de las consultas de la interfaz y combina documentos embebidos, referencias ObjectId, snapshots e índices especializados.

Indicador	Estado verificado	Lectura para la defensa
Motor	MongoDB 8.0.32 / WiredTiger	Servidor documental local
Modelo	9 colecciones lógicas	Una base con responsabilidades separadas
Datos	128 documentos lógicos	Escenario ficticio y reproducible
Índices	35 visibles	26 definidos por el proyecto, 8 _id y 1 automático time series
Consultas	15 guardadas en My Queries	Find, aggregate, lookup, GeoJSON y series temporales
Respaldo	mongodump restaurado y verificado	Se probaron documentos, metadatos e índices

Idea central

MongoDB no declara una colección principal. En este sistema viajes cumple ese papel porque concentra el patrón de acceso más importante: buscar una salida, validar el tramo, consultar disponibilidad y reservar un asiento.

Contenido

Resumen ejecutivo 2

Contenido . . . 3

1. MongoDB y el modelo documental 5

Modelo orientado al uso . . . 5

2. Estructura implementada 5

Arquitectura de la demostración . . . 6

Flujo principal . . . 6

Modelo documental implementado . . . 7

3. ObjectId y referencias 8

Relaciones reales del proyecto . . . 8

Error de tipo frecuente . . . 8

4. Viajes como agregado principal 8

Documento representativo . . . 9

Optimizaciones aplicadas . . . 9

Pares de venta . . . 9

Disponibilidad por tramo . . . 9

Reserva atómica . . . 10

5. Estrategia de índices 10

El índice principal . . . 11

Tipos destacados . . . 11

6. Rendimiento y uso de procesador 11

executionTimeMillis no es CPU puro . . . 11

Metodología de prueba . . . 12

Costo visible de los índices . . . 12

7. Tres consultas para la exposición 12

Consulta 1 - Viajes Mendoza-Córdoba . . . 12

Consulta 2 - Asientos libres por tramo . . . 13

Consulta 3 - Manifiesto mediante $lookup . . . 14

Consulta complementaria - Pasajes por DNI . . . 14

8. Reservas, pasajes y consistencia 15

Reserva y pasaje . . . 15

TTL parcial . . . 15

Validadores . . . 15

9. Serie temporal y geolocalización 15

Cuándo corresponde una time series . . . 16

10. Backup y restauración 16

Crear el respaldo . . . 16

Restaurar sobre la base . . . 16

Probar sin tocar la original . . . 16

Archivo comprimido . . . 16

mongoexport no reemplaza mongodump . . . 16

Política recomendada . . . 17

11. Guía breve para la defensa 17

Explicación sugerida del modelo . . . 17

Limitaciones conscientes . . . 17

12. Comandos de verificación 18

Referencias 18

1. MongoDB y el modelo documental

MongoDB almacena documentos BSON dentro de colecciones. Un documento se parece a un objeto JSON, pero BSON agrega tipos como ObjectId, Date, Decimal128, enteros de diferentes tamaños, datos binarios y expresiones regulares. El documento puede contener objetos y arreglos, por lo que una estructura jerárquica puede recuperarse en una sola lectura.

Concepto relacional	Equivalente documental	Aplicación en el proyecto
Base de datos	Base de datos	gestionpasajes
Tabla	Colección	viajes, pasajes, pasajeros
Fila	Documento	Un viaje programado
Columna	Campo	fechaSalida, estado
Clave primaria	_id	ObjectId del documento
Clave foránea	Referencia	viajeId, pasajeroId
JOIN	$lookup	Manifiesto de pasajeros

Modelo orientado al uso

En un diseño relacional suele buscarse normalización. En MongoDB el modelo se ajusta a la forma en que la aplicación lee y modifica los datos. Se puede duplicar información cuando esa decisión evita consultas repetidas, mantiene el historial o permite una operación atómica.

- Embeber cuando los datos pertenecen al agregado, se consultan juntos y tienen crecimiento acotado.
- Referenciar cuando la entidad tiene identidad independiente, se reutiliza o cambia por separado.
- Usar snapshots cuando una operación debe conservar el valor histórico aunque cambie la fuente.

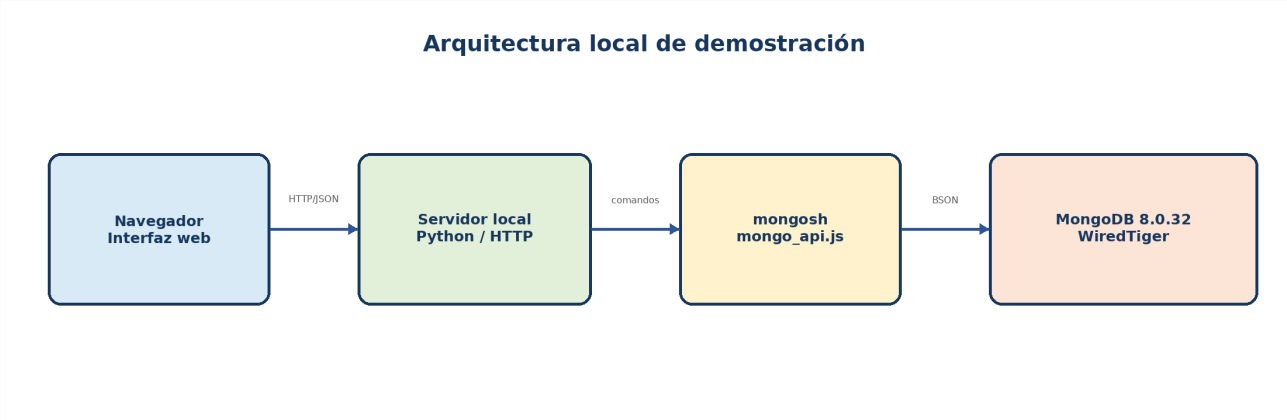
Punto de examen NoSQL no significa sin relaciones. El sistema utiliza relaciones mediante ObjectId y $lookup, pero MongoDB no impone claves foráneas. La aplicación y los scripts controlan esa integridad.

2. Estructura implementada

Colección	Docs.	Responsabilidad	Diseño destacado
usuarios	10	Cuenta, rol y empleado	Email único
pasajeros	15	Persona que viaja	Documento único y contacto
terminales	12	Terminal y ubicación	GeoJSON / 2dsphere
recorridos	6	Ruta, paradas y precios	Arreglos embebidos
colectivos	5	Vehículo y asientos	Patente e interno únicos
viajes	12	Salida e inventario	Colección operativa principal
reservas	9	Bloqueo temporal	TTL parcial
pasajes	31	Venta e historial	Pago y snapshots
seguimientoviajes	24	Telemetría	Time series

MongoDB también muestra `system.buckets.seguimientoviajes` y `system.views`. Son estructuras internas de la serie temporal, no colecciones de negocio, y no deben manipularse.

Arquitectura de la demostración

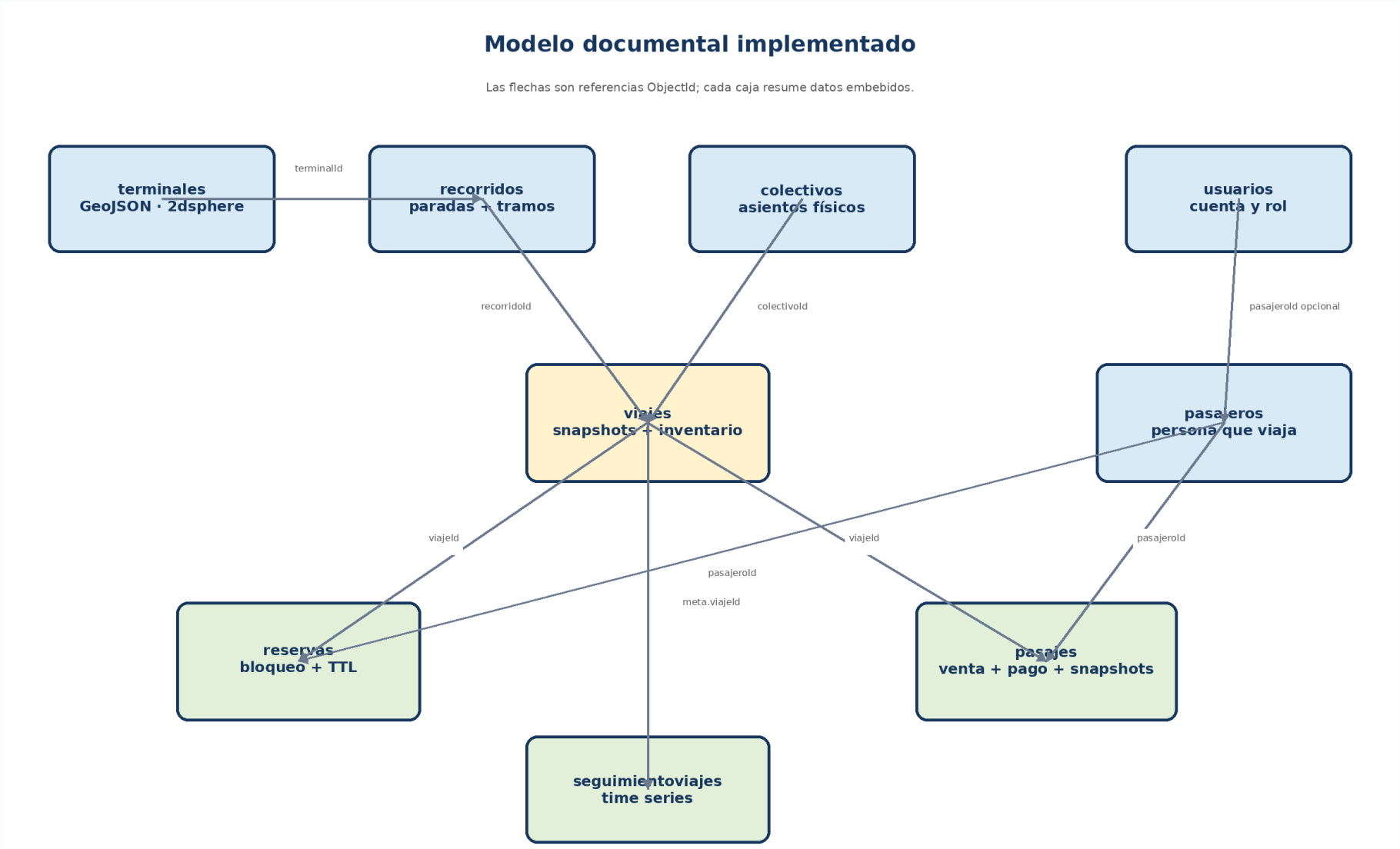


El navegador consume una API HTTP local implementada en Python. El servidor valida parámetros y ejecuta operaciones predefinidas mediante mongosh y mongo_api.js. La API no recibe JavaScript arbitrario desde el navegador.

Flujo principal

Paso	Operación	Colección dominante
1	Seleccionar origen, destino y fecha	terminales / viajes
2	Buscar salidas programadas	viajes
3	Calcular asientos libres por tramo	viajes.inventarioAsientos
4	Bloquear un asiento temporalmente	viajes + reservas
5	Confirmar venta y pago simulado	pasajes
6	Consultar historial o manifiesto	pasajes + pasajeros

Modelo documental implementado



Las flechas representan referencias ObjectId. Las cajas resumen datos embebidos y decisiones de modelado. Viajes concentra snapshots e inventario, mientras reservas y pasajes mantienen ciclos de vida independientes.

3. ObjectId y referencias

ObjectId es un tipo BSON de 12 bytes que MongoDB utiliza habitualmente para el campo _id. Se muestra como 24 caracteres hexadecimales y está pensado para generar identificadores con una probabilidad extremadamente baja de colisión en sistemas distribuidos.

```
ObjectId("66f500000000000000000001")
```

- _id identifica de forma única un documento dentro de la colección.
- MongoDB crea un índice único sobre _id en las colecciones normales.
- ObjectId conserva un orden temporal aproximado, pero no reemplaza un campo Date.
- No debe utilizarse como contraseña ni como mecanismo de autorización.

Relaciones reales del proyecto

Origen	Campo	Destino	Significado
viajes	recorridoId	recorridos._id	Recorrido base
viajes	colectivoId	colectivos._id	Vehículo asignado
reservas	viajeId	viajes._id	Salida reservada
reservas	pasajeroId	pasajeros._id	Persona que reserva
pasajes	viajeId	viajes._id	Salida vendida
pasajes	pasajeroId	pasajeros._id	Persona que viaja

Error de tipo frecuente

```
// Incorrecto: compara contra un string
db.viajes.find({ _id: "66f500000000000000000001" })

// Correcto: compara contra un ObjectId
db.viajes.find({
  _id: ObjectId("66f500000000000000000001")
})
```

Por qué no devuelve datos MongoDB compara valor y tipo. Un string con los mismos caracteres no es igual a un ObjectId.

4. Viajes como agregado principal

MongoDB no posee una propiedad llamada colección principal. En este sistema viajes cumple esa función por el patrón de acceso: buscar una salida, validar que el destino aparezca después del origen, mostrar el colectivo, calcular disponibilidad y reservar un asiento.

Documento representativo

```
{
  _id: ObjectId("66f500000000000000000001"),
  codigo: "VIA-2026-11015-01",
  recorridoId: ObjectId("66f300000000000000000001"),
  colectivoId: ObjectId("66f400000000000000000001"),
  conductorId: ObjectId("66f000000000000000000004"),
  fechaSalida: ISODate("2026-11-15T21:00:00Z"),
  estado: "programado",
  recorridoSnapshot: {
    codigo: "MDZ-CBA-01",
    paradas: [ /* MDZ, LUQ, VME, COR */ ],
    preciosPorTramo: [ /* importes Decimal128 */ ],
    paresVenta: ["MDZ|LUQ", "MDZ|VME", "MDZ|COR", ...]
  },
  colectivoSnapshot: {
    numeroInterno: "INT-101",
    patente: "AG123BC",
    capacidad: 46
  },
  inventarioAsientos: [
    { numero: 1, ocupaciones: [] },
    { numero: 2, ocupaciones: [ /* rangos ocupados */ ] }
  ]
}
```

Optimizaciones aplicadas

Decisión	Beneficio	Costo o cuidado
recorridoSnapshot	Evita lookup y conserva historial	Duplica datos del recorrido
colectivoSnapshot	Lectura directa del vehículo	No debe sobrescribirse el historial
paresVenta	Búsqueda directa origen-destino	Debe regenerarse si cambia el recorrido
inventarioAsientos embebido	Disponibilidad y reserva atómica	Aumenta el tamaño del documento
ocupaciones por rango	Permite revender un asiento en otro tramo	Requiere verificar superposición
índices por patrón de acceso	Reduce documentos examinados	Ocupa memoria y encarece escrituras

Pares de venta

Para el recorrido MDZ - LUQ - VME - COR se precalculan todas las combinaciones válidas donde el origen aparece antes que el destino:

```
[
  "MDZ|LUQ", "MDZ|VME", "MDZ|COR",
  "LUQ|VME", "LUQ|COR", "VME|COR"
]
```

La consulta puede buscar MDZ|COR directamente. No necesita recorrer las paradas de cada viaje para comprobar orden y existencia.

Disponibilidad por tramo

Cada ocupación utiliza un intervalo de paradas. Dos intervalos se superponen cuando se cumplen ambas condiciones:

```
ocupacion.desdeOrden < tramoSolicitado.hastaOrden
ocupacion.hastaOrden > tramoSolicitado.desdeOrden
```

Ejemplo Un pasaje MDZ-LUQ ocupa el segmento 0-1. El mismo asiento puede venderse LUQ-COR porque el segundo tramo comienza cuando termina el primero.

Reserva atómica

La aplicación utiliza updateOne con una condición de disponibilidad y arrayFilters. MongoDB agrega la ocupación solamente si el asiento continúa libre. Si dos clientes compiten por el mismo asiento, una sola actualización puede cumplir la condición. Si falla la inserción de la reserva, la aplicación ejecuta una compensación y retira el bloqueo recién agregado.

5. Estrategia de índices

Un índice es una estructura ordenada que permite localizar candidatos sin recorrer toda la colección. Un plan con COLLSCAN examina documentos; un plan con IXSCAN recorre claves del índice y recupera los documentos necesarios.

Balance Los índices aceleran lecturas, pero consumen disco y memoria. Cada inserción, eliminación o actualización de un campo indexado también debe mantener esas estructuras.

Colección	Índice	Tipo / objetivo
usuarios	uq_usuarios_email	Único por email
pasajeros	uq_pasajeros_documento	Compuesto y único
terminales	uq_terminales_codigo	Único por código
terminales	ix_terminales_ubicacion	2dsphere
recorridos	uq_recorridos_codigo	Único por código
recorridos	ix_recorridos_terminales	Multikey sobre paradas
colectivos	uq_colectivos_numeroInterno	Único
colectivos	uq_colectivos_patente	Único
viajes	uq_viajes_codigo	Único
viajes	ix_viajes_fecha_estado	Calendario
viajes	ix_viajes_terminales_fecha	Terminal y fecha
viajes	ix_viajes_colectivo_fecha	Agenda del colectivo
viajes	ix_viajes_parVenta_estado_fecha	Búsqueda principal multikey
viajes	ix_viajes_conductor_fecha	Agenda del conductor
reservas	uq_reservas_codigo	Único
reservas	ix_reservas_viaje_estado_vencimiento	Disponibilidad
reservas	ix_reservas_pasajero_fecha	Historial
reservas	ttl_reservas_pendientes_venceEn	TTL parcial
pasajes	uq_pasajes_codigo	Único
pasajes	ix_pasajes_viaje_estado	Manifiesto
pasajes	ix_pasajes_pasajero_fecha	Historial
pasajes	ix_pasajes_venta_fecha_canal	Ventas
pasajes	ix_pasajes_documento_pasajero	Búsqueda por DNI
pasajes	ix_pasajes_comprador_fecha	Compras del usuario
pasajes	ix_pasajes_recorrido_venta_fecha	Recaudación por recorrido
seguimientoviajes	ix_seguimiento_viaje_fecha	Viaje y rango temporal

El índice principal

```
{
  "recorridoSnapshot.paresVenta": 1,
  estado: 1,
  fechaSalida: 1
}
```

El orden sigue el patrón igualdad, igualdad y rango: primero el par origen-destino, después el estado y por último la fecha. ParesVenta es un arreglo, por lo que MongoDB crea un índice multikey. Los otros campos del índice son escalares.

Tipos destacados

Tipo	Ejemplo del proyecto	Función
Único	uq_pasajeros_documento	Impide duplicados
Compuesto	parVenta + estado + fecha	Resuelve un patrón completo
Multikey	recorridoSnapshot.paresVenta	Indexa cada elemento del arreglo
2dsphere	terminales.ubicacion	Consultas geográficas
TTL parcial	reservas.venceEn	Elimina pendientes vencidas
Time series	meta.viajeId + fecha	Rangos temporales

6. Rendimiento y uso de procesador

Explain muestra el plan y el trabajo de una consulta. Las métricas esenciales son nReturned, totalKeysExamined, totalDocsExamined, executionTimeMillis, winningPlan e indexName.

```
db.viajes.find({
  "recorridoSnapshot.paresVenta": "MDZ|COR",
  estado: "programado",
  fechaSalida: { $gte: ISODate("2026-11-15T00:00:00Z") }
}).explain("executionStats")
```

Medición actual	Con índice	COLLSCAN forzado	Interpretación
nReturned	3	3	Mismo resultado
totalKeysExamined	3	0	COLLSCAN no usa índice
totalDocsExamined	3	12	El índice reduce el trabajo
executionTimeMillis	aprox. 4 ms	aprox. 0 ms	Muestra demasiado pequeña

Lectura correcta

Que COLLSCAN marque 0 ms no demuestra que sea mejor. Con 12 documentos dominan la resolución del reloj, la caché y el costo inicial. La evidencia escalable es examinar 3 documentos en lugar de 12.

executionTimeMillis no es CPU puro

La duración observada puede combinar trabajo de CPU, espera de disco, caché, bloqueos, planificación y carga concurrente. Para observar el proceso mongod en Linux se pueden utilizar:

```
pidstat -p "$(pgrep -n mongod)" 1
top -p "$(pgrep -n mongod)"
mongotop 1
```

mongotop informa tiempo de lectura y escritura por namespace. El profiler de MongoDB registra operaciones lentas, planes y documentos examinados, pero agrega carga y debe activarse con cuidado.

```
db.setProfilingLevel(1, { slowms: 50 })
db.system.profile.find().sort({ ts: -1 }).limit(5)
db.setProfilingLevel(0)
```

Metodología de prueba

- Cargar un volumen representativo, no solamente 12 viajes.
- Repetir las consultas para separar caché fría y caliente.
- Comparar operaciones equivalentes y registrar promedio o percentiles.
- Revisar claves y documentos examinados, no solamente milisegundos.
- Medir también escrituras, CPU, memoria y disco.
- Evitar conclusiones basadas en una sola ejecución.

Costo visible de los índices

Colección	Docs.	Datos lógicos	Tamaño medio	Índices
viajes	12	34.755 B	2.896 B	225.280 B
pasajes	31	25.531 B	823 B	245.760 B
reservas	9	3.198 B	355 B	159.744 B
pasajeros	15	5.819 B	387 B	73.728 B

En esta base pequeña, las páginas mínimas de cada índice explican que ocupen más que los datos. No es una proyección directa del comportamiento con millones de documentos.

7. Tres consultas para la exposición

Consulta 1 - Viajes Mendoza-Córdoba

```
db.viajes.find(
  {
    "recorridoSnapshot.paresVenta": "MDZ|COR",
    estado: "programado",
    fechaSalida: {
      $gte: ISODate("2026-11-15T00:00:00Z"),
      $lt: ISODate("2026-11-18T00:00:00Z")
    }
  },
  {
    codigo: 1,
    fechaSalida: 1,
    fechaLlegadaEstimada: 1,
    "colectivoSnapshot.numeroInterno": 1,
    "colectivoSnapshot.capacidad": 1
  }
).sort({ fechaSalida: 1 })
```

Parte	Función
paresVenta = MDZ|COR	Origen válido antes del destino
estado = programado	Descarta viajes no vendibles
$gte / $lt	Rango de fechas semiabierto
proyección	Devuelve solamente datos necesarios
sort	Orden cronológico
índice	ix_viajes_parVenta_estado_fecha
resultado actual	3 viajes

Consulta 2 - Asientos libres por tramo

```
db.viajes.aggregate([
  {
    $match: {
      _id: ObjectId("66f500000000000000000001"),
      estado: "programado"
    }
  },
  {
    $project: {
      _id: 0,
      codigo: 1,
      asientosDisponibles: {
        $map: {
          input: {
            $filter: {
              input: "$inventarioAsientos",
              as: "asiento",
              cond: {
                $eq: [
                  {
                    $size: {
                      $filter: {
                        input: "$$asiento.ocupaciones",
                        as: "ocupacion",
                        cond: {
                          $and: [
                            { $lt: ["$$ocupacion.desdeOrden", 3] },
                            { $gt: ["$$ocupacion.hastaOrden", 0] },
                            {
                              $or: [
                                { $in: ["$$ocupacion.tipo", ["pasaje", "bloqueo"]] },
                                {
                                  $and: [
                                    { $eq: ["$$ocupacion.tipo", "reserva"] },
                                    { $gt: ["$$ocupacion.venceEn", "$$NOW"] }
                                  ]
                                }
                              ]
                            }
                          ]
                        }
                      }
                    }
                  },
                  0
                ]
              }
            }
          },
          as: "libre",
          in: "$$libre.numero"
        }
      }
    }
  },
  { $set: { cantidadDisponible: { $size: "$asientosDisponibles" } } }
])
```

$match selecciona el viaje por _id. El primer $filter recorre los asientos. El segundo busca ocupaciones superpuestas. $size cuenta conflictos, $eq exige cero y $map devuelve el número de cada asiento libre. Para MDZ-COR, desdeOrden es 0 y hastaOrden es 3.

Resultado actual	El viaje de ejemplo posee 42 asientos disponibles para el tramo completo Mendoza-Córdoba.

Consulta 3 - Manifiesto mediante $lookup

```
db.pasajes.aggregate([
  {
    $match: {
      viajeId: ObjectId("66f500000000000000000001"),
      estado: { $in: ["confirmado", "utilizado"] }
    }
  },
  {
    $lookup: {
      from: "pasajeros",
      localField: "pasajeroId",
      foreignField: "_id",
      as: "pasajeroActual"
    }
  },
  { $unwind: "$pasajeroActual" },
  {
    $project: {
      _id: 0,
      pasaje: "$codigo",
      asiento: "$asientoNumero",
      pasajero: {
        $concat: ["$pasajeroActual.apellido", ", ", "$pasajeroActual.nombre"]
      },
      documento: "$pasajeroActual.numeroDocumento",
      origen: "$tramo.origen",
      destino: "$tramo.destino",
      embarque: "$embarque.estado"
    }
  },
  { $sort: { asiento: 1 } }
])
```

Etapa	Lectura
$match	Filtra pasajes del viaje y usa ix_pasajes_viaje_estado
$lookup	Relaciona pasajes.pasajeroId con pasajeros._id
$unwind	Convierte el arreglo pasajeroActual en un objeto
$project	Da formato y concatena nombre completo
$sort	Ordena por asiento
Resultado	4 pasajeros

Consulta complementaria - Pasajes por DNI

```
db.pasajes.find({
  "pasajeroSnapshot.tipoDocumento": "DNI",
  "pasajeroSnapshot.numeroDocumento": "40111222"
})
```

El snapshot evita buscar primero al pasajero para obtener su _id. El índice ix_pasajes_documento_pasajero permite responder directamente al patrón de la interfaz.

8. Reservas, pasajes y consistencia

Reserva y pasaje

Reserva	Pasaje
Bloqueo temporal	Venta confirmada
Puede vencer	Conserva historial
Estado pendiente/confirmada/cancelada	Estado confirmado/cancelado/utilizado
Tiene venceEn	Tiene venta y pago
Usa TTL para pendientes	No expira automáticamente

pasajes.reservaId puede contener un ObjectId si la compra nació de una reserva. Un valor null representa una venta directa, por ejemplo en boletería.

TTL parcial

```
db.reservas.createIndex(
  { venceEn: 1 },
  {
    name: "ttl_reservas_pendientes_venceEn",
    expireAfterSeconds: 0,
    partialFilterExpression: { estado: "pendiente" }
  }
)
```

El monitor TTL elimina en segundo plano las reservas pendientes cuyo venceEn ya pasó. La eliminación puede tener una demora breve. El TTL no limpia automáticamente la ocupación embebida en viajes; por eso la consulta ignora reservas vencidas y existe una operación de limpieza.

Validadores

Las colecciones utilizan $jsonSchema para controlar campos obligatorios, tipos BSON, valores permitidos y estructuras embebidas. La flexibilidad documental no obliga a aceptar documentos inconsistentes.

```
estado: {
  enum: ["pendiente", "confirmada", "vencida", "cancelada"]
}
```

Límite de los validadores Pueden comprobar estructura y tipos, pero no verifican automáticamente que un ObjectId referenciado exista en otra colección.

9. Serie temporal y geolocalización

seguimientoviajes registra mediciones append-only. fecha es el timeField y meta agrupa la identidad del viaje y del colectivo. MongoDB organiza internamente estas mediciones en buckets para comprimirlas y optimizar rangos temporales.

```
{
  fecha: ISODate("2026-11-15T21:15:00Z"),
  meta: {
    viajeId: ObjectId("66f500000000000000000001"),
    viajeCodigo: "VIA-2026-11015-01",
    colectivoInterno: "INT-101"
  },
  ubicacion: {
    type: "Point",
    coordinates: [-68.1, -32.7]
  },
  velocidadKmh: 82,
  pasajerosEstimados: 41
}
```

Cuándo corresponde una time series

- Se insertan mediciones ordenadas por tiempo.
- Las consultas usan rangos temporales.
- Los registros viejos casi no se modifican.
- La compresión y las agregaciones temporales resultan importantes.

Por qué reservas no es time series Una reserva cambia de estado y tiene un ciclo de vida transaccional. Corresponde una colección normal con índice TTL, no una serie de mediciones inmutables.

10. Backup y restauración

mongodump genera un respaldo BSON masivo. Los archivos .bson conservan documentos y los .metadata.json conservan opciones, validadores, índices y configuración de colecciones.

Crear el respaldo

```
mongodump \
  --db gestionpasajes \
  --out backups/backup_2026-10-27
```

Restaurar sobre la base

```
mongorestore \
  --drop \
  --nsInclude="gestionpasajes.*" \
  backups/backup_2026-10-27
```

Cuidado --drop elimina cada colección antes de restaurarla. Debe usarse solamente cuando se desea reemplazar el contenido existente.

Probar sin tocar la original

```
mongorestore \
  --nsFrom="gestionpasajes.*" \
  --nsTo="gestionpasajes_prueba.*" \
  backups/backup_2026-10-27
```

Archivo comprimido

```
mongodump \
  --db gestionpasajes \
  --archive=gestionpasajes_2026-10-27.archive.gz \
  --gzip

mongorestore \
  --archive=gestionpasajes_2026-10-27.archive.gz \
  --gzip \
  --drop
```

mongoexport no reemplaza mongodump

mongoexport	mongodump
Intercambio JSON/CSV	Respaldo BSON
Normalmente una colección	Base o colecciones completas
No conserva todo el entorno	Conserva metadatos e índices
Compatible con mongoimport	Compatible con mongorestore

Política recomendada

- Definir frecuencia y retención.
- Mantener al menos una copia fuera del servidor principal.
- Proteger archivos y credenciales.
- Comprobar integridad y realizar restauraciones de prueba.
- Definir RPO: cuánto dato puede perderse.
- Definir RTO: cuánto puede tardar la recuperación.

Estado del proyecto El respaldo final del 23/10/2026 se restauró en una base temporal. Se verificaron documentos, tipos de colección, índices y la restauración de seguimientoviajes como time series.

11. Guía breve para la defensa

Pregunta probable	Respuesta breve
¿Por qué MongoDB?	Porque el viaje contiene estructuras jerárquicas que se consultan y actualizan como un agregado.
¿Por qué viajes es principal?	La búsqueda, disponibilidad y reserva comienzan en una salida programada.
¿Por qué duplicar datos?	Para reducir lecturas y conservar historia mediante snapshots controlados.
¿Cómo se relacionan colecciones?	Mediante ObjectId y, cuando corresponde, $lookup.
¿Cómo evita doble reserva?	Con un updateOne condicional atómico sobre el inventario embebido.
¿Cómo demuestra el índice?	Explain muestra IXSCAN, 3 claves y 3 documentos examinados.
¿Por qué el tiempo no alcanza?	La base es pequeña; importa la reducción de trabajo y una prueba repetida a escala.
¿Qué hace el TTL?	Elimina en segundo plano reservas pendientes vencidas.
¿El backup está probado?	Sí, se restauró en una base temporal independiente.

Explicación sugerida del modelo

El sistema utiliza un modelo documental porque los viajes concentran recorrido, paradas, colectivo e inventario de asientos. Combinamos documentos embebidos, snapshots y referencias según las consultas de la interfaz. La disponibilidad por tramos se resuelve dentro del viaje, mientras reservas, pasajes y pasajeros conservan vida independiente mediante ObjectId.

Limitaciones conscientes

- La interfaz es una demostración local sin autenticación productiva ni TLS.
- Los datos son ficticios y el volumen sirve para demostrar funcionalidad, no carga real.
- MongoDB no verifica referencias entre colecciones como una clave foránea SQL.
- Los snapshots requieren una política clara: el historial se conserva y los derivados se regeneran.
- My Queries pertenece al perfil local de Compass y no se incluye en mongodump; el proyecto aporta un instalador reproducible.

12. Comandos de verificación

```
# Verificación completa del modelo e índices
mongosh gestionpasajes/scripts/12_verificacion_completa.js

# Verificación de las consultas guardadas
mongosh gestionpasajes/scripts/13_verificacion_myqueries.js

# Iniciar la interfaz
cd gestionpasajes/interfaz
python3 server.py
```

La verificación actual informa VERIFICACION_COMPLETA_OK y MY_QUERIES_VERIFICADAS_OK. La interfaz queda disponible en http://127.0.0.1:4173 con MongoDB activo en 127.0.0.1:27017.

Referencias

- Zapata, Fernando. Clase MongoDB Modelado, material de Diseño de Bases de Datos II.
- Zapata, Fernando. Clase MongoDB Operación, material de Diseño de Bases de Datos II.
- Zapata, Fernando. Clase MongoDB Índices y rendimiento, material de Diseño de Bases de Datos II.
- Zapata, Fernando. Clase MongoDB Backup, material de Diseño de Bases de Datos II.
- Proyecto gestionpasajes: scripts de estructura, operaciones, consultas y verificación.
- Respaldo final gestionpasajes, verificación de restauración del 23 de octubre de 2026.

Repositorio https://github.com/franbujaldon/gestionpasajes